

Machine Learning

From first principles to Generative AI

A concept-only study guide for client interviews

Build the foundations in the order they depend on one another: data and probability, classical ML, neural networks, representation learning, Transformers, and reliable GenAI systems.

This guide assumes no prior ML knowledge. It explains ideas before equations, uses practical examples, and connects ordinary ML decisions to LLM and RAG applications. Programming exercises and console labs are deliberately deferred.

Prepared for Roy | 8 October 2026

What you should be able to do after studying

- Explain how a model learns and why evaluation can be misleading.
- Choose a baseline, a split strategy, and a metric for a business problem.
- Explain neural networks, attention, LLM training, and fine-tuning without relying on buzzwords.
- Distinguish prompting, RAG, fine-tuning, and training from scratch.
- Discuss model limitations, project decisions, and production tradeoffs clearly.

How to use this guide: read a session, close the guide, explain its key idea aloud, and answer its checkpoint. Revisit weak answers before adding new topics.

This is a learning and interview preparation resource. The image-classification example is an educational scenario, not a validated medical system.

UNDERSTAND / EXPLAIN / CONNECT / RECALL

Build accurate explanations before moving into implementation.

Your concept-only study plan

Use two focused days if possible. If you have one full day, complete Sessions 1-7 first and use the remaining sessions as a second pass. The times are planning estimates, not mastery guarantees.

Session	Concepts	Reading pages	Suggested time
1	AI map, math intuition, ML workflow	3-6	50-70 min
2	Data, preprocessing, splits, leakage	7-8	45-60 min
3	Generalization and evaluation	9-11	60-80 min
4	Classical algorithms and model choice	12-15	70-90 min
5	Optimization and neural networks	16-19	70-90 min
6	CNNs, sequence models, generative models	20-22	50-70 min
7	Tokens, embeddings, attention, Transformers	23-25	70-90 min
8	LLM training, adaptation, inference	26-28	60-80 min
9	RAG, evaluation, production design	29-32	70-90 min
10	Case studies and debugging	33-35	45-60 min
11	Interview answers and closed-notes test	36-41	60-90 min
12	Cheat sheet, glossary, gap review	42-44	25-40 min

For each session

1. Read for understanding. Explain every bold term in your own words.
2. Connect the concept to one example: spam detection, price prediction, MRI classification, or a document assistant.
3. Close the PDF and answer the checkpoint aloud in 45-90 seconds.
4. Mark each topic: can explain / uncertain / cannot explain. Revisit the last two categories.

Priority for an interview in seven days: data leakage; metrics; overfitting; regression versus classification; trees and ensembles; gradient descent; neural network training; embeddings; attention; RAG versus fine-tuning. Read advanced details after these become clear.

A reliable answer structure: definition, how it works, example, tradeoff. For a design question: requirements, baseline, data, evaluation, deployment, failure handling.

1. The AI and ML landscape

Artificial intelligence (AI) is the broad effort to build systems that perform tasks associated with intelligence. Some AI systems use explicit rules; others learn from data.

Machine learning (ML) learns statistical patterns from examples. Instead of writing every rule for spam detection, we provide messages and outcomes, then learn a function that predicts an outcome for a new message.

Deep learning (DL) uses neural networks with multiple processing layers to learn representations. It is especially useful for complex inputs such as images, audio, and language. More layers do not automatically make a model better.

Generative AI (GenAI) produces content such as text, images, audio, or code. Modern GenAI commonly uses deep learning, but generative modeling is a wider statistical idea. An LLM is one family of generative models; GenAI is not limited to LLMs.

Learning setting	Available signal	Example
Supervised	Input and target pairs	Predict spam from labeled email
Unsupervised	Inputs without task labels	Group similar customers
Self-supervised	Targets derived from the data	Predict a text's next token
Semi-supervised	Small labeled set plus unlabeled data	Classify documents with limited annotation
Reinforcement	Rewards from actions and outcomes	Learn a control policy

Regression predicts numerical quantities, such as delivery time. **Classification** predicts categories, such as spam or not spam. **Clustering** discovers groups; those groups are not automatically meaningful business categories.

Model, algorithm, and system are different. An algorithm is a learning procedure. A trained model contains learned parameters. A production system also includes preprocessing, APIs, storage, permissions, monitoring, and user interaction.

GenAI connection: next-token training is usually self-supervised; instruction tuning often uses supervised examples; preference-based post-training can use reinforcement learning or other preference objectives.

Checkpoint: Why can a text model learn from unlabeled documents? Explain where its training targets come from.

2. Mathematical intuition: vectors and change

You need the meaning of these tools before detailed derivations.

Scalar: one number, such as a temperature. **Vector:** an ordered collection of numbers, such as three customer features. **Matrix:** a rectangular collection of numbers, such as a dataset with rows for examples and columns for features. **Tensor:** a general multidimensional array; an image batch can have batch, height, width, and channel dimensions.

Shape matters. If a dataset has n examples and d features, its feature matrix has shape n by d . A linear model's weight vector has d entries. Multiplying them produces one score per example. Shape mismatches often reveal a misunderstanding of what a dimension represents.

Dot product: multiply corresponding vector entries and add the results. For $x = [2, 3]$ and $w = [4, 1]$, $x \cdot w = 11$. It measures weighted contribution and, geometrically, relates to alignment and magnitude.

Norm: measures vector size. The L2 norm is Euclidean length; the L1 norm sums absolute entries. **Cosine similarity** compares directions: dot product divided by the product of lengths. It is undefined if either vector has zero length. Similar direction can indicate related content when the embedding model is appropriate.

Derivative: describes how one quantity changes as another changes. **Partial derivative:** changes one input while holding others fixed. **Gradient:** collects partial derivatives for all parameters; it points toward locally fastest increase of a differentiable function.

Chain rule: tracks how an upstream change passes through a chain of computations. Backpropagation uses it to calculate how each weight affects the final loss.

Worked intuition: if a model predicts a price that is too high, the gradient tells us which weights contributed to that error and how changing them locally affects the loss. We update opposite the gradient to try to reduce it.

GenAI connection: embeddings are vectors; attention uses matrix operations; neural network training depends on gradients. A vector's coordinate is not necessarily a human-readable concept such as happiness or urgency.

Checkpoint: What is the difference between an embedding vector, its length, and cosine similarity?

3. Mathematical intuition: probability and uncertainty

Probability describes uncertainty about an outcome. **Conditional probability**, $P(A \text{ given } B)$, describes uncertainty after observing B . A language model estimates the distribution of the next token given the preceding tokens.

Bayes' rule updates a belief using evidence: $P(A \text{ given } B) = P(B \text{ given } A) \times P(A) / P(B)$, when $P(B)$ is nonzero. The prior is the initial belief; the likelihood describes how compatible the evidence is with a hypothesis; the posterior is the updated belief.

Mean is the arithmetic average; **median** is the middle value after sorting. The median is often less sensitive to extreme observations. **Variance** describes squared spread around the mean; **standard deviation** is its square root and uses the original units.

Correlation describes association, not causation. A model may discover that umbrella sales predict rain, but buying umbrellas does not cause rain. Spurious shortcuts can work in training and fail when the environment changes.

Likelihood treats observed data as fixed and asks how well parameter choices explain it. Maximizing likelihood is equivalent to minimizing negative log-likelihood. Logarithms turn products of probabilities into sums, making optimization more convenient.

Entropy measures uncertainty in a distribution. A distribution concentrated on one outcome has lower entropy than one spread over many outcomes. **Cross-entropy** penalizes assigning low probability to the correct outcome. In binary classification, predicting 0.99 for a positive example receives a smaller penalty than predicting 0.01.

Softmax converts a set of real-valued scores into positive values summing to one. They are useful as modeled class probabilities, but they are not automatically well calibrated. A model can be confidently wrong.

GenAI connection: LLMs produce scores called logits, transform them into a token distribution, and select or sample a token. A high token probability means the continuation fits the model's learned distribution; it does not prove a factual claim is true.

Checkpoint: Explain why a fluent, confident answer can still be incorrect without saying that the model simply searches a database.

4. The complete ML workflow

Start with a decision. What will the prediction change? Who uses it? What costs more: a missed positive, a false alarm, latency, or manual review? A business success metric and a model score serve different purposes.

1. **Frame the task:** define the input, target, prediction time, and acceptable failure behavior.
2. **Audit data:** inspect provenance, consent, schema, missingness, label quality, class balance, and coverage.
3. **Design the split:** hold out examples that represent genuinely new production situations.
4. **Explore training data:** study distributions and likely shortcuts; keep final test data isolated from model decisions.
5. **Build a baseline:** majority class, historical mean, simple rules, or a small interpretable model.
6. **Preprocess and train:** fit transformations and model parameters using the training portion.
7. **Validate and tune:** compare models, features, hyperparameters, and decision thresholds.
8. **Evaluate the frozen choice:** use the held-out test set and report limitations and uncertainty.
9. **Deploy and monitor:** track input quality, prediction quality, latency, cost, drift, and business outcomes.

Parameters are learned, such as a neural weight or a linear coefficient. **Hyperparameters** configure learning, such as learning rate, tree depth, or regularization strength. Tuning a hyperparameter using the test set compromises its independence.

Training changes model parameters. **Inference** uses a trained model on new inputs. Training may require labels; inference must use only information actually available at prediction time.

Example: for predicting employee attrition next quarter, termination paperwork created after departure is an invalid input. It would not exist when the prediction is needed.

GenAI connection: choosing a foundation model does not remove this workflow. You still need a task definition, representative evaluation data, a baseline prompt, versioned changes, operational monitoring, and an escalation path.

Checkpoint: If the business wants fewer missed fraudulent transactions, what should you clarify before selecting a metric or model?

5. Data preparation and feature engineering

Features are model inputs; a **label** is the target. Feature engineering converts available data into useful signals, such as order frequency or time since the last purchase. A feature must be available at the moment of prediction.

Missing data: investigate why values are missing. Options include removing unsuitable rows or columns, imputing values, and adding missingness indicators. A missing salary can carry different information from a salary of zero. Fit imputation statistics on training data only.

Categorical encoding: one-hot encoding represents categories with indicator columns; ordinal encoding preserves a real order when one exists. Arbitrary integer codes can falsely imply that one city is larger or closer to another. Target encoding uses label statistics and needs careful out-of-fold handling to avoid leakage.

Scaling: standardization commonly subtracts the training mean and divides by the training standard deviation. Min-max scaling maps using the training minimum and maximum; unseen values may fall outside the chosen interval. Unit-norm normalization rescales each vector's length. The word normalization is used inconsistently, so specify the operation.

When scaling matters: distance methods, SVMs, regularized linear models, and neural optimization often benefit. Decision trees usually do not need it because splits depend on ordered feature values rather than Euclidean distance.

Outliers: check whether they are errors, rare valid events, or the phenomenon you want to detect. Automatically removing rare events can destroy a fraud dataset.

Imbalance: use appropriate metrics, class weights, threshold tuning, or resampling. Apply resampling only to training folds. Synthetic sampling before a split can put closely related examples on both sides.

GenAI connection: document cleaning, preserving headings, deduplicating content, retaining metadata, and choosing chunks are feature preparation for retrieval. An embedding cannot repair an incorrectly extracted table.

Checkpoint: Why should a scaler be fitted on training data but reused to transform validation and test data? See references [1], [2].

6. Splits, cross-validation, and leakage

Training set: fits parameters. **Validation set:** helps choose models and configurations. **Test set:** estimates the performance of the final selected approach on untouched data. There is no universally correct 80/20 split; size, dependence, and deployment conditions matter.

K-fold cross-validation: repeatedly trains on k-1 folds and validates on the remaining fold. Aggregate scores to understand performance variation. Fit preprocessing independently inside each training fold. Cross-validation does not rescue an invalid split strategy.

Situation	Appropriate split principle
Independent labeled examples	Random split; stratify when appropriate
Several records per patient or user	Keep each person within one partition
Predict future events	Train on the past; validate on later periods
Deployment to unseen sites	Consider holding out sites or locations

Leakage is information entering training or model selection that would not be available in the intended prediction setting. Typical forms include fitting a scaler on all data, features created after the outcome, duplicate images across partitions, and repeatedly choosing changes after checking the test score.

Worked example: 100 patients each contribute 20 MRI slices. Randomly splitting the 2,000 slices can place nearly identical slices from the same patient in both training and test sets. The evaluation then partly measures recognition of familiar patients. A patient-level split better evaluates generalization to new patients.

After tuning: freeze decisions, then evaluate once on the untouched test set. If you continue adjusting after observing its results, use a new holdout for an independent estimate. Nested cross-validation is an option for more rigorous model-selection evaluation.

GenAI connection: an LLM can have benchmark contamination from pretraining. For RAG and fine-tuning, deduplicate semantically similar questions across evaluation partitions and hold out realistic queries. An answer source may legitimately be in a RAG corpus; the evaluation answer and test-derived tuning decisions should not leak into training or prompts.

Checkpoint: Why does stratifying a slice-level MRI split fail to solve patient leakage? See references [1], [2].

7. Generalization, bias, variance, and regularization

Generalization is useful performance on new examples from the intended operating environment. Memorizing training examples is insufficient.

Underfitting: the model fails to capture important patterns; both training and validation performance are poor. Causes include an overly simple model, poor features, or insufficient training. **Overfitting:** the model learns noise or overly specific patterns; training performance improves while validation performance is much worse.

Bias describes systematic approximation error. **Variance** describes sensitivity to changes in the training sample. A shallow tree can have high bias; a very deep tree can have high variance. This is a useful classical framework, not a universal law that every neural network obeys in a simple curve.

Regularization discourages undesirable complexity. L2 penalizes squared weights; L1 penalizes absolute weights and can produce sparse coefficients. Stronger penalties may reduce variance while introducing bias. Weight decay is related to L2 regularization, but the equivalence depends on the optimizer and implementation.

Other controls: more representative data, data augmentation, early stopping, reduced capacity, dropout, and good features. Augmentation creates label-preserving variations; rotating an image is useful only when orientation changes do not change the label.

Learning curves: plot training and validation results as training size or training progress changes. A persistent generalization gap suggests overfitting; poor performance on both sets may suggest underfitting or a data problem. Diagnose before adding layers.

Distribution shift: production data differs from training data. A good random-split result can still fail on a new hospital, language, customer group, or time period.

GenAI connection: fine-tuning on a small repetitive dataset can overfit its phrasing or harm broader capabilities. A document assistant can perform well on familiar questions but fail on ambiguous queries or new document formats.

Checkpoint: If training accuracy is 99% and validation accuracy is 72%, list three hypotheses and the evidence you would seek before changing the model.

8. Classification metrics and thresholds

Choose the positive class explicitly. In spam detection, a positive is spam. **TP** is correctly detected spam; **FP** is legitimate mail wrongly flagged; **FN** is missed spam; **TN** is correctly accepted legitimate mail.

Metric	Formula	What it answers
Accuracy	$(TP + TN) / \text{all examples}$	How often is the class correct?
Precision	$TP / (TP + FP)$	Among predicted positives, how many are correct?
Recall / sensitivity	$TP / (TP + FN)$	Among actual positives, how many are found?
Specificity	$TN / (TN + FP)$	Among actual negatives, how many are rejected correctly?
F1	$2PR / (P + R)$	How are precision and recall balanced?

Worked example: TP=80, FP=20, FN=10, TN=890. Accuracy is 97%; precision is 80%; recall is about 88.9%; F1 is about 84.2%. The apparently excellent accuracy hides the false alarms and missed cases. Formulas need a defined convention when their denominators are zero.

Threshold: a score becomes a class after comparing it with a cutoff. Lowering a threshold generally increases or preserves recall while increasing or preserving false-positive count on the same data. Select the threshold using validation data and business costs; 0.5 is not inherently optimal.

ROC-AUC measures ranking across thresholds using true-positive and false-positive rates.

Precision-recall curves are often more informative when positives are rare. PR-AUC and average precision summarize these curves differently; name the statistic you report.

Calibration: among examples assigned a probability near 0.8, roughly 80% should be positive if predictions are well calibrated. Ranking quality and calibration are separate. Log loss penalizes confident wrong probabilities; accuracy evaluates thresholded labels.

Multiclass: macro averaging treats each class equally; weighted averaging weights by class support; micro averaging pools counts. Include per-class results because an average can hide minority-class failure.

GenAI connection: evaluate classifiers in a moderation or routing system with these metrics. Evaluate generated answers separately; an LLM's self-reported confidence is not a calibrated probability of truth.

Checkpoint: Which would you prioritize when missed fraud is costly, and what tradeoff would you discuss? See [3].

9. Regression and clustering evaluation

For regression, define the error e as predicted value minus actual value. The sign tells you whether the prediction is too high or too low.

Metric	Meaning	Main tradeoff
MAE	Average absolute error	Original units; less sensitive to extreme errors than MSE
MSE	Average squared error	Strong penalty for large errors; squared units
RMSE	Square root of MSE	Original units; still emphasizes large errors
R-squared	$1 - \text{residual sum of squares} / \text{total sum of squares}$	Performance relative to predicting the evaluation-set mean

Worked example: actual values [10,20,30], predictions [12,18,33]. Errors are [2,-2,3]. MAE=7/3, or 2.33. MSE=17/3, or 5.67. RMSE is about 2.38. Explain what these units mean for the user.

R-squared can be negative. It means the predictions are worse, under this measure, than using the evaluation-set mean. An R-squared of 0.8 does not mean 80% of individual predictions are correct. Constant targets require special handling because the denominator is zero.

MAPE divides error by actual values. It becomes problematic near zero and can produce misleading comparisons. Choose a metric suited to the scale and business impact.

Clustering: when labels are unavailable, silhouette score summarizes separation and cohesion using a chosen distance. A good score does not prove that clusters are useful. If reference groupings exist, adjusted Rand index or other agreement measures can help. Business validation still matters.

Beyond one number: inspect error distributions and slices: large versus small orders, new versus returning users, languages, hospitals, and time periods. Report sample counts. A metric on 12 examples is much less informative than one on a representative large holdout.

GenAI connection: response latency, cost, retrieval rank, and groundedness measure different aspects of a system. Do not collapse them into a single quality score without stating the tradeoffs.

Checkpoint: Why might a client prefer MAE to RMSE, and when would emphasizing large errors be desirable? See [3].

10. Linear and logistic regression

Linear regression predicts a number using a weighted sum of features plus an intercept: predicted $y = w \cdot x + b$. Each coefficient describes the model's change in prediction per unit of a feature, holding the others fixed. Correlated features complicate coefficient interpretation.

Ordinary least squares minimizes squared residuals. Important concerns include nonlinear relationships, influential outliers, correlated inputs, and changing relationships. You can add nonlinear features while remaining linear in the learned coefficients.

Ridge adds an L2 penalty; **Lasso** adds an L1 penalty; **Elastic Net** combines them. Scale features when comparing or regularizing coefficients. These models are useful baselines when tabular data is limited and interpretability matters.

Logistic regression is a classifier despite its name. For a binary target, it computes a linear score then passes it through sigmoid: $p = 1 / (1 + \exp(-\text{score}))$. Training commonly minimizes binary cross-entropy. A threshold turns the modeled probability into a class.

What is linear here? The log-odds are linear in the inputs. The probability is not a straight line because sigmoid is nonlinear. Without engineered features, the decision boundary is linear in feature space. Multinomial logistic regression extends the idea with softmax.

Strengths: fast, understandable, and a strong baseline for text classification with suitable sparse features. **Limitations:** it may miss complex interactions unless feature engineering captures them. Regularization strength, class weights, and threshold are important choices.

Interview distinction: linear regression typically predicts a continuous quantity; logistic regression models class probabilities. A classifier's output can be numerical without making the task a regression problem.

GenAI connection: the final output projection in a language model also produces scores that become probabilities, but the preceding nonlinear neural network is far richer than ordinary logistic regression.

Checkpoint: If logistic regression returns 0.7, what does the threshold decide, and what would calibration tell you?

11. Trees, forests, and boosting

Decision tree: repeatedly divides feature space using questions such as income below a threshold. Classification splits commonly reduce Gini impurity or entropy; regression splits can reduce squared error. Leaf predictions summarize examples reaching that leaf.

Strengths: captures interactions, handles nonlinear decision boundaries, usually does not need scaling, and can be explained when small. **Limitations:** deep trees overfit and can change considerably after a small data change. Control maximum depth, minimum leaf size, and pruning.

Bagging: train multiple models on bootstrap samples and aggregate their predictions. A bootstrap sample draws training examples with replacement. Aggregating suitably diverse models can reduce variance.

Random forest: combines bootstrapped trees with random feature subsets considered at splits. Classification commonly aggregates tree class probabilities; regression averages predictions. Random feature selection decorrelates trees. Tune tree count, depth, minimum leaf size, and feature sampling.

Boosting: adds learners sequentially to improve the current ensemble. Gradient boosting fits new learners to loss-related residual signals, specifically negative gradients. Learning rate, number of trees, tree complexity, and regularization interact. A smaller learning rate often needs more trees.

XGBoost is an optimized gradient-boosted tree implementation with regularization and efficient training. It is not a neural network and not a synonym for every boosting method.

Method	Main idea	Typical concern
Single tree	Recursive feature splits	High variance when deep
Random forest	Aggregate diverse trees	Larger model; less direct interpretation
Gradient boosting	Sequential loss correction	Sensitive tuning; overfitting if unchecked

Feature importance caution: impurity-based importance can favor high-cardinality features. Correlated features complicate any attribution. Importance describes model usage, not proof of causality.

GenAI connection: a GenAI application can still use a boosted-tree model for fraud checks, routing, or risk scoring. An LLM is not automatically the best tool for structured prediction.

Checkpoint: Why is a random forest usually more stable than a single deep tree? How does boosting differ from bagging?

12. KNN, SVM, and Naive Bayes

K-nearest neighbors (KNN) stores examples and predicts from nearby training points. For classification it uses neighbors' labels; for regression it uses their values, often with distance weighting. The distance measure defines what nearby means.

Choose k: small k is sensitive to noise; larger k smooths boundaries but can underfit. Scale features so one large numerical range does not dominate distance. Prediction can be expensive on large datasets. In high dimensions, nearest and farthest distances can become less distinctive: one aspect of the curse of dimensionality.

Support vector machine (SVM) seeks a boundary with a large margin. Support vectors are influential training points near the boundary. A soft margin allows some errors; C controls the penalty for violations relative to margin size.

Kernel trick: compute similarities corresponding to an implicit transformed feature space without explicitly materializing that space. An RBF kernel can capture nonlinear structure; gamma controls locality. SVMs often need scaling and can become expensive with large datasets. Their scores are not automatically calibrated probabilities.

Naive Bayes combines class priors and feature likelihoods using Bayes' rule, with a simplifying conditional-independence assumption: given the class, features are treated as independent. It can work well for text even when the assumption is imperfect. Smoothing avoids assigning zero likelihood to unseen features.

Model	Good starting use	Watch for
KNN	Small datasets; meaningful distance	Scaling, dimension, inference cost
SVM	Moderate datasets; margin separation	Kernel settings and training cost
Naive Bayes	Sparse text classification	Independence assumptions and calibration

GenAI connection: embedding retrieval also finds nearby representations, but it returns evidence rather than necessarily classifying a query. Approximate nearest-neighbor search trades some retrieval exactness for speed. KNN and a vector database are related through distance search, not identical systems.

Checkpoint: Why can Euclidean distance give misleading neighbors when one feature is salary and another is age?

13. Clustering, PCA, and representations

K-means groups points around k centroids by repeatedly assigning each point to its nearest centroid and updating centroids to cluster means. It minimizes within-cluster squared Euclidean distances. Initialization matters because the procedure can converge to different local solutions.

Limitations: choose k ; scale inputs; be careful with outliers and irregularly shaped or differently sized clusters. K-means is most naturally suited to compact groups under its distance objective. A cluster number is an identifier, not a semantic label.

DBSCAN groups dense neighborhoods and can mark isolated points as noise. It can discover irregular shapes and does not require k , but neighborhood radius and minimum sample settings matter. Varying density and high dimensions can make it difficult.

Principal component analysis (PCA) finds orthogonal directions capturing large variance in centered data. The first component captures the greatest variance; later components capture remaining variance subject to orthogonality. Reducing dimensions can compress data and remove some noise.

What PCA does not guarantee: maximum variance is not maximum predictive relevance. A low-variance direction may be important for classification. Standardize when differing units would dominate. Fit PCA on training data only. Principal components are usually less directly interpretable than original features.

Feature selection versus extraction: selection keeps some original features; extraction constructs new representations. PCA is extraction. A learned embedding is also a representation, but unlike ordinary PCA it can be nonlinear and trained for a task.

GenAI connection: embeddings represent text so that useful relationships are reflected in geometry. Clustering embeddings can organize documents, while PCA can help compress or visualize them. A two-dimensional visualization may distort distances; apparent separation is not proof of model quality.

Checkpoint: Why is PCA described as unsupervised, and why might the first principal component be unhelpful for predicting a label?

14. Loss functions and optimization

Loss function: quantifies prediction error for an example or batch. **Objective:** may combine data loss with regularization. **Optimizer:** decides parameter updates. **Evaluation metric:** measures the outcome you care about; it need not be the function optimized during training.

Gradient descent updates parameters opposite the loss gradient: $\text{new weight} = \text{old weight} - \text{learning rate} \times \text{gradient}$. This is a local step; it does not guarantee the global optimum of a neural network's nonconvex objective.

Batch gradient descent uses the entire training set for one update. **Stochastic gradient descent** can use one example; common practice uses mini-batches. A mini-batch balances compute efficiency with noisy but frequent updates.

Learning rate: too large can cause unstable or divergent training; too small can slow progress or leave the model poorly optimized. A schedule changes it during training. **Momentum** accumulates a moving direction of updates. **Adam** uses moving estimates of gradients and squared gradients to adapt updates; AdamW uses decoupled weight decay.

Epoch: one pass through the training dataset. **Batch size:** examples per update. **Step:** one parameter update. With 1,000 examples and batch size 100, one epoch contains 10 steps, assuming no dropped or accumulated batches. Gradient accumulation can simulate a larger effective batch using multiple smaller batches.

Loss choices: MSE is common for regression; binary cross-entropy for binary classification; categorical cross-entropy for mutually exclusive classes. Multi-label classification differs because several labels can be true at once, often using separate sigmoid outputs.

Why not optimize accuracy directly? The usual thresholded accuracy does not give useful smooth gradients. A differentiable surrogate can guide learning, while validation accuracy or another business metric assesses the selected model.

GenAI connection: next-token training typically uses cross-entropy over the vocabulary. Optimizing this objective teaches plausible continuations; it does not directly certify factual truth or user satisfaction.

Checkpoint: Distinguish loss, optimizer, and metric using one classification example. See [4].

15. Neural network building blocks

Artificial neuron: computes a weighted sum plus bias, then applies an activation: $\text{output} = \text{activation}(w \cdot x + b)$. Weights control contributions; a bias shifts the response. This is a mathematical component, not a faithful simulation of a biological neuron.

Layer: a collection of transformations. A dense layer connects each output to all inputs from the preceding layer. **Hidden layers** transform representations before the output layer. **Depth** counts successive transformations; **width** concerns the number of units or representation dimensions.

Why activations matter: stacking only linear or affine layers still yields one affine transformation. Nonlinear activations let the network represent complex relationships.

Activation	Main idea	Practical concern
ReLU	$\max(0, x)$	Units can stop activating for observed inputs
Sigmoid	Maps to 0-1	Saturation can produce small gradients
Tanh	Maps to -1 to 1	Also saturates
GELU	Smoothly gates values	Common in Transformer feed-forward layers
Softmax	Normalizes class scores	Used across a chosen class or token axis

Choose the output for the task. A price model can use an unrestricted numerical output. A binary classifier can use sigmoid. A mutually exclusive multiclass classifier can use softmax. The activation and loss must agree with how the task is formulated.

Forward propagation: send an input through layers to obtain outputs. Early layers may learn simpler transformations; later layers compose them. These learned features are not guaranteed to align neatly with human concepts.

Capacity and data: adding layers increases expressive power but may increase compute, tuning difficulty, and overfitting. A small tabular dataset may favor a linear or tree-based model.

GenAI connection: Transformer feed-forward blocks are neural networks, and token embeddings and attention projections contain learned weights. An LLM is much more than an attention formula alone.

Checkpoint: Why cannot ten stacked linear layers, without nonlinear operations, represent arbitrary nonlinear behavior?

16. Backpropagation and the training loop

Backpropagation efficiently calculates loss gradients through a computational graph using the chain rule. **Gradient descent** uses those gradients to update parameters. Backpropagation and optimization are separate stages.



Forward computation, then backward gradients, then an optimizer step

1. **Forward:** compute predictions from a training batch.
2. **Loss:** compare predictions with target outputs.
3. **Backward:** calculate how each trainable parameter affects the loss.
4. **Update:** the optimizer applies an update, then gradients are cleared or accumulated according to the training procedure.
5. **Repeat and validate:** monitor performance on data that does not participate in the weight updates.

Simple intuition: if a weight slightly increases an already overestimated prediction, its gradient can indicate that decreasing the weight locally reduces error. In a deep network, the effect travels through many intermediate operations, which is why the chain rule is essential.

Vanishing gradients: gradients shrink through successive computations, making early layers hard to train. **Exploding gradients:** gradients grow excessively, causing instability. Activation choices, initialization, normalization, residual connections, and gradient clipping can help in suitable settings.

Initialization: starting every unit identically can preserve symmetry and prevent diverse features from developing. Practical schemes choose random scales suited to the network's activations and structure.

Autodifferentiation: frameworks calculate derivatives of implemented operations. They do not decide whether your labels, split, loss, or scientific assumptions are valid.

GenAI connection: a language model's loss flows back through output projections, Transformer blocks, and eligible embeddings. With a frozen backbone and trainable adapters, gradients update the adapters rather than every original weight.

Checkpoint: During inference, do you normally run backpropagation? Explain what changes when training a model instead. See [4].

17. Stable neural training and evaluation

Dropout randomly disables a subset of activations during training, commonly with rescaling. This discourages reliance on particular units. Standard evaluation turns dropout off. A deliberate uncertainty method may use it at inference, but that is a separate choice.

Batch normalization uses batch statistics during training and typically learned running statistics during inference. **Layer normalization** normalizes within each example across a specified set of feature dimensions; it does not depend on batch statistics in the same way. They are not interchangeable by name.

Early stopping: stop or select a checkpoint when validation performance stops improving under a chosen rule. It uses validation data, not the final test set. Save the best checkpoint rather than assuming the last epoch is best.

Training mode versus evaluation mode: frameworks change behavior of modules such as dropout and batch normalization. Disabling gradient tracking is a separate operation from switching mode; both can matter for efficient, correct inference.

Data augmentation: generate task-valid variations of training inputs. Text augmentation can accidentally change sentiment, factual claims, or labels. Medical-image augmentation must preserve relevant anatomy and interpretation. Never assume more augmentation is always better.

Reproducibility: record data versions, preprocessing, split identifiers, model configuration, random seeds, libraries, and checkpoints. Seeds help control randomness but do not guarantee identical results across all hardware and operations.

Read training traces: unstable loss can indicate a large learning rate, bad scaling, corrupted data, or numerical issues. Low training loss with poor validation results can indicate overfitting, split shift, leakage-related selection, or preprocessing mismatch. Investigate the pipeline before tuning blindly.

GenAI connection: fine-tuning also needs checkpoint selection, held-out task evaluation, and regression checks on broader capabilities. Lower fine-tuning loss alone does not prove a better assistant.

Checkpoint: Why can evaluating a model in training mode give inconsistent or misleading predictions?

18. CNNs and transfer learning for images

Convolutional neural network (CNN): applies learned filters across spatial locations. Weight sharing lets the same filter detect a pattern in different locations and uses fewer parameters than a fully connected treatment of every pixel pair.

Kernel: a learned local filter. **Feature map:** its spatial responses. **Channels:** distinct input or output feature dimensions. An RGB image has three input channels; deeper channels are learned features.

Stride: movement between filter placements. **Padding:** extends boundaries to control output size and edge coverage. **Receptive field:** the input region that can influence a particular feature. Stacking layers expands effective spatial context.

Output size intuition: for one spatial dimension with input N , padding P , kernel K , stride S , and dilation 1, $\text{output} = \text{floor}((N + 2P - K) / S) + 1$. Thus 32 with kernel 3, padding 1, stride 1 stays 32.

Pooling: reduces spatial resolution by summarizing neighborhoods. Max pooling chooses a maximum; average pooling computes a mean. It can make small shifts less disruptive while discarding detail. **Global average pooling** summarizes each entire feature map and often reduces parameters relative to flattening into a large dense layer.

Transfer learning: reuse representations learned on a source task. A frozen backbone plus a new output head is feature extraction. Unfreezing some or all backbone layers and training further is fine-tuning. Often begin with a small learning rate when adapting a pretrained backbone.

Example: an MRI classifier may adapt a pretrained image network, but natural-image pretraining does not guarantee suitability for medical images. Audit channel handling, normalization, patient-level splits, class distribution, and external generalization.

GenAI connection: transfer learning is the same broad principle behind adapting pretrained language models. Modern image generation can also use Transformers; CNNs are not the only architecture for images.

Checkpoint: Explain kernel, stride, pooling, and transfer learning as four different concepts.

19. Sequence models and the route to Transformers

Sequence: an ordered set of inputs, such as words, audio frames, or sensor readings. Order changes meaning: dog bites person differs from person bites dog.

Recurrent neural network (RNN) updates a hidden state as it reads one element after another. It carries information through the sequence, but long dependencies can be difficult to learn and time-step dependence limits parallel training.

LSTM and GRU use gates to control what information is retained, updated, or exposed. They help address long-term dependency problems but do not eliminate every limitation of recurrent computation.

Encoder-decoder: the encoder produces a representation of input; the decoder produces an output sequence. Early sequence-to-sequence designs compressed input into a fixed representation, creating a bottleneck on long inputs.

Attention: lets a computation use a weighted combination of relevant representations rather than relying on one fixed summary. A translation decoder can focus on different source tokens as it produces different output tokens.

Self-attention: a sequence attends to representations from that same sequence.

Cross-attention: queries come from one set of representations while keys and values come from another, such as a decoder consulting encoded input.

Transformer: uses attention and feed-forward transformations as central building blocks, with position information and normalization. It supports parallel processing of token positions during conventional training. Ordinary autoregressive generation still produces new tokens sequentially, though implementations can accelerate parts of the process.

Tradeoff: standard full self-attention compares many token pairs and can have quadratic sequence-length cost for the attention computation and matrix. Efficient attention implementations and alternative architectures change practical cost profiles; avoid claiming every language model has identical complexity.

GenAI connection: a decoder-style Transformer is common for next-token language generation. An encoder-style model can be excellent for representations or classification. Sequence processing and text generation are not the same task.

Checkpoint: What bottleneck does attention address, and why does parallel training not imply fully parallel autoregressive generation?

20. Discriminative and generative modeling

Discriminative modeling focuses on predicting a target given an input, often modeling $P(y \text{ given } x)$ or directly learning a decision boundary. Logistic regression predicts a class from features.

Generative modeling models how data is distributed, such as $P(x)$ or the joint distribution $P(x,y)$. A conditional generative model learns a distribution of outputs given context, such as an image given a text prompt. Learning $P(\text{output given context})$ can still be generative when the aim is to produce rich outputs.

Important nuance: the presence of a conditional probability alone does not decide the category. An LLM generates a sequence by modeling conditional next-token distributions; Naive Bayes is traditionally a generative classifier because it models class priors and feature likelihoods. A model family and a product's role should be discussed explicitly.

Family	Core idea	Typical tradeoff
Autoregressive	Generate one element conditioned on preceding elements	Sequential generation can add latency
Variational autoencoder	Learn a latent distribution and reconstruct samples	Useful latent structure; reconstruction can lose detail
GAN	Generator competes with discriminator	Sharp outputs; unstable training or mode collapse
Diffusion	Learn to reverse a controlled noising process	Strong sample quality; iterative generation cost

Autoencoder: compresses an input to a representation and reconstructs it. A basic autoencoder is not automatically a good sampler of new realistic data. A VAE adds a probabilistic latent formulation and a regularization objective encouraging a usable latent distribution.

GAN mode collapse: the generator produces too little diversity, even if some samples look realistic. **Diffusion:** during training, examples are corrupted with noise and a network learns a suitable denoising or related prediction objective. Sampling follows a learned reverse process. Latent diffusion works in a compressed representation. See [8], [9].

GenAI connection: an LLM and an image diffusion model are both generative, but their training objectives and generation procedures differ.

Checkpoint: Why is GenAI a broader category than chatbot technology?

21. Tokenization and embeddings

Tokenization converts text into units and identifiers. Tokens may be words, subword fragments, punctuation, bytes, or combinations determined by the tokenizer. A word can span several tokens; token count depends on language, spelling, and tokenizer.

Vocabulary: the set of token identifiers known to a tokenizer. **Token embedding:** a learned vector associated with an identifier. **Contextual representation:** a later representation influenced by surrounding tokens, so the same word can take different meanings in different contexts.

Static versus contextual: an initial embedding lookup gives the same vector for an identifier in a given model. Transformer layers then produce context-sensitive representations. In bank of a river and bank loan, surrounding words help distinguish meanings.

Document or sentence embeddings: encode a larger text into a vector for tasks such as semantic retrieval. A model specifically trained for retrieval often works better than naively averaging arbitrary hidden states of a generative model. Its training objective defines which relationships it emphasizes.

Similarity: cosine compares direction; dot product also depends on magnitude unless vectors are normalized. Euclidean distance measures straight-line separation. Use the metric expected by the embedding model and index configuration.

Representation compatibility: vectors from unrelated embedding models do not share a meaningful coordinate system merely because their dimensions match. Changing models generally requires re-embedding the corpus and validating retrieval.

Context window: the input and generated token span that a model can process under its serving rules. It is not permanent memory and not a guarantee of equal attention to every part of a long prompt.

GenAI connection: an embedding model retrieves useful evidence; a generative model writes the response. A single architecture can support several roles, but their objectives and evaluation differ.

Checkpoint: Why is an embedding vector useful for finding related text, and why is it not a compressed database record that can be exactly decoded?

22. Attention: queries, keys, and values

Query (Q): the representation seeking relevant information. **Key (K):** a representation used to compare relevance. **Value (V):** the information mixed according to the resulting attention weights. These are learned projections, not literal search strings, database keys, or dictionary values.



Learned projections turn token representations into weighted information

For a set of token representations X , learned matrices produce $Q = XW_q$, $K = XW_k$, and $V = XW_v$. Scaled dot-product attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^{\text{transpose}} / \sqrt{d_k}) V$$

Step by step: dot products compare queries with keys; division by the square root of key dimension controls score scale; softmax normalizes scores over allowed positions; the result weights and combines values. A mask can block disallowed positions before softmax.

Example: while interpreting the token it in the animal was tired because it ran, an attention head may place useful weight on representations related to animal. This illustrates information mixing; it is not a guarantee that attention weights provide a complete or faithful explanation of a model's reasoning.

Multi-head attention: multiple learned projection sets provide different representation subspaces. Their outputs are combined. Different heads can capture different relationships, but do not assume each has a clean human-assigned role.

Causal masking: a token cannot attend to future tokens when trained for next-token prediction. In teacher-forced training, many positions can be processed together while masks prevent future information from leaking into each prediction.

Self-attention obtains Q , K , and V from the same sequence representation. **Cross-attention** obtains queries from one representation and keys/values from another. Decoder-only language models commonly use causal self-attention.

GenAI connection: attention mixes available context; it does not guarantee retrieval of facts absent from that context or eliminate hallucinations.

Checkpoint: What would go wrong if a next-token model could attend to the future target tokens during training? See [5].

23. The Transformer architecture

Input representation: token embeddings plus some mechanism to encode position or relative order. Attention alone, without suitable order information or structural constraints, does not adequately distinguish all token orders. Position methods include absolute embeddings and relative or rotary approaches.

A typical block contains: multi-head attention, a position-wise feed-forward network, residual connections, and layer normalization. Their ordering differs among architectures. Residual paths preserve a direct route for information and gradients; feed-forward networks transform features at each position.

Architecture	Visibility / role	Common use
Encoder-only	Often bidirectional input context	Classification, token labeling, embeddings
Decoder-only	Causal context for ordinary generation	Autoregressive text completion
Encoder-decoder	Encode input, decode output with cross-attention	Translation and conditional generation

Language-model output: project a position's hidden representation to vocabulary logits, then use a probability transformation and a decoding rule. A Transformer can be used for non-generative tasks too; architecture and training objective are different concepts.

Training versus generation: training on a known sequence can compute losses across positions in parallel with the correct mask. At generation time, the next token becomes part of the context for the following step. This explains both training efficiency and generation latency.

Position and long context: the ability to accept a long sequence does not guarantee strong retrieval of every detail. Evaluate the actual tasks, positions of evidence, distractors, and input lengths you expect.

What makes it large? Learned parameter count, training data, and compute can scale capabilities, but size alone does not determine suitability. Smaller specialized models can meet a task with lower latency or cost.

GenAI connection: you can explain an LLM as a trained sequence model that transforms context into a distribution over continuations. This is more precise than saying it simply predicts words or understands everything.

Checkpoint: Distinguish tokenization, positional information, self-attention, and the feed-forward network. See [5].

24. How an LLM is trained and adapted

Pretraining: learn broad representations and language patterns from a large corpus. In causal language modeling, training predicts the next token from earlier tokens. Targets come from the text itself, making the objective self-supervised.

Teacher forcing: supply the correct earlier tokens during training, rather than always feeding the model's own sampled previous outputs. This supports efficient training but differs from the conditions of free-running generation.

Instruction tuning / supervised fine-tuning (SFT): train on examples of instructions and desirable responses. It teaches task and response behavior. Dataset quality, diversity, and consistency matter more than simply accumulating many near-duplicate examples.

Preference training: compare possible responses and favor desirable behavior. A classic RLHF pipeline trains a reward model from preference rankings and uses reinforcement learning to improve a policy. Other methods, such as direct preference optimization, optimize preference objectives without that same explicit reward-model-and-RL loop.

Alignment: attempts to improve behavior according to intended requirements, preferences, or constraints. It is not proof that a model is always truthful, safe, or consistent. A reward or preference signal is a proxy and can be imperfect. See [6].

Continued pretraining: further train on domain text with a language-model objective. This differs from teaching explicit question-answer behavior through SFT. Both change weights and require evaluation for unwanted regressions.

Inference context: a prompt or retrieved passage influences the current computation without normally changing weights. In-context learning is useful adaptation within a context, not equivalent to parameter updates or permanent learning.

GenAI connection: task reliability requires more than pretraining. Good data, adaptation, retrieval, constrained interfaces, and system-level evaluation all contribute.

Checkpoint: Compare pretraining, SFT, preference training, and adding a passage to a prompt. Which normally change parameters?

25. Fine-tuning, PEFT, LoRA, and quantization

Full fine-tuning updates most or all model parameters on a target objective. It can be costly in compute, optimizer state, and deployment storage. It may change behavior broadly, including harming capabilities you did not intend to alter.

Parameter-efficient fine-tuning (PEFT) updates a smaller set of parameters. Examples include adapters and low-rank updates. It reduces trainable state, but base-model weights, activations, and training computations still consume resources.

LoRA: keep a pretrained weight matrix frozen and learn a low-rank update. Conceptually, effective $W = W_0 + BA$, often with a scaling factor. The matrices A and B have a small inner rank r . Tune adapter rank, target modules, learning rate, and dataset choices. Lower rank reduces trainable parameters but is not guaranteed to match full fine-tuning on every task. See [7].

Quantization: represent weights or computations using lower numerical precision to reduce memory and often improve efficiency, with possible quality or hardware tradeoffs. It is a representation technique, not itself a training objective.

QLoRA: combines a quantized frozen base with trainable low-rank adapters under its particular training method. Low-precision storage and adapter training are distinct ideas; do not claim all arithmetic is performed directly in four-bit integers. See [10].

When to fine-tune: recurring behavior, format, style, or domain-task performance remains inadequate after a good baseline; you have enough high-quality examples and an independent evaluation set. Fine-tuning is not a reliable substitute for up-to-date factual retrieval.

Avoid: training on private information without an appropriate data policy, small repetitive datasets without validation, evaluating only training loss, or promising that fine-tuning removes hallucinations.

GenAI connection: adapting a language model and adapting a CNN share transfer-learning principles. The main differences are architecture, objective, data scale, and system constraints.

Checkpoint: Explain why LoRA saves trainable state while still requiring the base model during training and inference.

26. Generation, decoding, and inference

Autoregressive generation: compute a next-token distribution, choose a token, append it, and repeat until stopping. **Logits** are pre-normalization scores; a decoding rule selects the output.

Greedy decoding chooses the highest-scoring token at each step. It is simple but does not guarantee the globally most probable sequence. **Beam search** keeps several candidate continuations; it can be useful for some sequence tasks but is not automatically ideal for conversational text.

Temperature rescales logits before sampling. Lower temperature concentrates the distribution; higher temperature spreads it. A temperature approaching zero corresponds conceptually to selecting maxima, but exact API behavior varies. Lower temperature does not guarantee factuality or perfect reproducibility.

Top-k restricts sampling to a chosen number of high-scoring tokens. **Top-p / nucleus sampling** keeps a probability-mass-based candidate set. These are decoding controls, not model retraining. Some model interfaces do not expose all controls.

Context length and output budget: longer retrieved context consumes space and compute. Reserve room for the response. Truncating blindly can remove instructions, evidence, or important qualifiers.

KV cache: stores previously calculated keys and values to avoid recomputing them for every generation step. It speeds decoding but uses memory that grows with active sequences and context. Batching improves throughput, while individual latency depends on workload and serving choices.

Structured outputs: a schema or constrained decoder can improve syntactic validity. Valid JSON does not imply correct facts, and a syntactically valid tool argument can still be inappropriate. Validate semantics and permissions before execution.

GenAI connection: track time to first token, total response latency, tokens per second, and cost separately. A fast first token does not imply fast completion of a long answer.

Checkpoint: What changes when you adjust temperature, and why is it not a fix for missing factual evidence?

27. RAG: retrieval-augmented generation

RAG combines retrieval of external information with generation conditioned on that information. Modern application pipelines often use a separately trained retriever and an existing generator; they need not jointly train the two as in some research formulations. See [11].



Ingestion builds the index; each query retrieves permitted evidence

Offline / ingestion path

1. Parse documents and preserve source identifiers, headings, dates, and permissions.
2. Split into meaningful chunks; avoid separating a rule from its exceptions or a table from its labels.
3. Compute embeddings and store them with text and metadata in a searchable index.

Online / query path

1. Authenticate the user; determine which documents are permitted.
2. Retrieve candidate passages using lexical, dense, or hybrid search.
3. Rerank if useful, remove redundancy, and fit the context budget.
4. Ask the generator to answer from evidence with source references and an explicit missing-evidence behavior.
5. Validate the answer, return source links, and log appropriate diagnostics.

Chunking tradeoff: small chunks improve focused retrieval but may lose context; large chunks preserve context but add irrelevant tokens. Overlap can preserve continuity while increasing storage and repeated evidence. Select using retrieval and answer evaluation, not a universal chunk size.

Hybrid search: lexical retrieval helps exact identifiers and rare terms; dense retrieval helps semantic paraphrases. **Reranking:** a second model scores candidate query-passage pairs more carefully, often at extra cost.

Failure modes: wrong source version, inaccessible document, poor OCR, missed evidence, irrelevant chunks, misleading synthesis, or fabricated citations. Adding more retrieved text can worsen a response if it adds conflicts or distractors.

ML connection: retrieval has its own representation, ranking, and evaluation problem. The generator cannot reliably compensate for evidence that never reached it.

Checkpoint: If an answer is wrong, how do you distinguish a retrieval failure from a generation failure?

28. Prompting versus RAG versus fine-tuning

Approach	What changes	Best starting use	Main limitation
Prompting	Current input instructions and examples	Clarify task, format, and constraints	Context capacity; inconsistent behavior
RAG	Evidence supplied at inference	Current, private, attributable facts	Retrieval and source-quality failures
Fine-tuning	Learned parameters or adapters	Repeated task behavior, style, format	Data and training cost; regression risk
Training from scratch	Entire learned foundation	Specialized requirements and very large resources	Usually excessive for an application

A sensible decision sequence: define representative queries and a scoring rubric; establish a baseline; improve instructions and examples; add retrieval if missing or changing knowledge is the gap; consider fine-tuning when recurring behavior still fails and suitable training examples exist.

Few-shot prompting: place a few demonstrations in context. **Zero-shot prompting:** describe the task without demonstrations. Neither normally updates model weights. Explicit delimiters can help distinguish instructions from supplied data, but they are not a complete security boundary.

Use combinations: a tuned model can retrieve documents; a RAG system still needs clear prompts. These approaches solve overlapping but different problems rather than forming a strict ranking of better technologies.

Company policy example: a leave-policy assistant needs current authorized policy documents with citations. Start with RAG. If its answers repeatedly violate a required response structure despite clear prompting, supervised adaptation may help, but it should still retrieve current policy facts.

Text classification example: for millions of short messages and fixed categories, compare a small supervised classifier, an embedding-based model, and an LLM. A lower-cost model may meet requirements better.

GenAI connection: prompt engineering is part of system design, not a substitute for data quality, evaluation, access control, or deployment engineering.

Checkpoint: A client asks for fine-tuning to keep a chatbot aware of daily policy changes. What would you propose, and why?

29. Evaluating GenAI and retrieval

Build a held-out evaluation set from representative user tasks. Include simple questions, multi-document questions, ambiguous requests, missing evidence, conflicting versions, exact identifiers, and disallowed access attempts. Separate development queries from final evaluation.

Retrieval metrics: recall@k measures whether relevant items are found among the first k results, with a defined relevance set. Precision@k measures how many retrieved items are relevant. MRR emphasizes the rank of the first relevant result; nDCG accounts for graded relevance and ranking position. These metrics require relevance judgments.

Answer metrics: correctness, groundedness, completeness, relevance, citation support, instruction following, and appropriate abstention. A factual answer may still be unsupported by the supplied source; a perfectly grounded answer may quote an outdated policy.

Perplexity: exponential average negative log-likelihood on token prediction. Lower can indicate better prediction on the evaluated text distribution. It does not directly measure helpfulness, reasoning correctness, or factual reliability. Comparisons require compatible tokenization and evaluation setup.

Automatic evaluation: exact match works for some short answers; overlap metrics can miss valid paraphrases or reward wrong text sharing similar words. Task-specific checks, such as executing code tests, can be more meaningful when appropriate.

LLM-as-judge: useful for scaling rubric-based review, but judges can show bias, inconsistency, or susceptibility to misleading outputs. Calibrate against human-reviewed examples, blind comparisons where possible, and track disagreement. Do not assume the evaluator is ground truth.

Operational metrics: total latency, time to first token, error rate, cost per successful task, escalation rate, and user outcomes. Choose a quality-versus-cost target rather than optimizing tokens alone.

ML connection: held-out evaluation, error analysis, slice analysis, and baseline comparison apply just as strongly to GenAI. Repeatedly tuning on a fixed test set eventually overfits the evaluation process.

Checkpoint: Why can recall@5 improve while answer quality becomes worse?

30. Production, security, and monitoring

Version the full pipeline: data, feature logic, embedding model, index, foundation model, prompt, adapters, evaluation set, and application code. A retrieval change can alter answers even when the LLM is unchanged.

Monitor drift: data drift concerns changing input distributions; concept drift concerns changing relationships between inputs and outcomes. Track performance when labels arrive, not just input histograms. In GenAI, stale documents or changed user tasks can degrade quality.

Prompt injection: malicious or misleading instructions in user content, retrieved documents, or tool outputs try to redirect the application. Treat external content as untrusted data; enforce permissions in application code rather than relying on the model to decide authorization.

Access control: authenticate before retrieval and filter evidence by permitted scope. Rerankers, logs, caches, and generated responses must not expose unauthorized information. Permission checks should precede sensitive content entering the model context.

Tool use: the model proposes a structured action; the application validates arguments, permission, and state before execution. Use bounded retries, timeouts, audit trails, and appropriate human review for consequential actions. A tool call is not proof that an action succeeded; inspect its result.

Reliability: handle rate limits, dependency failures, malformed outputs, empty retrieval, and unavailable models. Plan fallback responses and escalation. Cache only when correctness, freshness, and permission boundaries are preserved.

Privacy: minimize sensitive content sent to models and retained in logs. Understand provider data handling, tenancy, retention, and the organization's requirements. Those depend on the actual deployment; avoid blanket claims about every API.

Explainability: use suitable attribution or interpretability tools for the model and task. A plausible natural-language explanation generated by an LLM is not automatically a faithful account of internal computations.

Checkpoint: Why is a prompt saying do not reveal private files insufficient to enforce document-level authorization?

31. Case study: an MRI image classifier

Use this as a reasoning template for your MRI project. Only describe architecture choices, results, and contributions that you actually implemented; the design below is illustrative.

Problem: classify images into clearly defined dataset categories. Clarify whether predictions are per slice, scan, or patient. A high score on a classroom dataset is not evidence of clinical usefulness.

Data: identify source, licensing, number of patients and images, class counts, labeling process, and preprocessing. Audit duplicates and whether filenames, borders, or acquisition settings reveal the target.

Split: group by patient when identifiers exist. Consider a held-out site or acquisition period. If patient identifiers are unavailable, explicitly report that leakage cannot be fully ruled out rather than claiming perfect independence.

Baseline and model: compare a reasonable small baseline with transfer learning. If using a pretrained backbone, explain why it was selected: available pretrained weights, data size, compute budget, and validation results. Do not say it is best simply because it is famous.

Training: document image resizing, channel conversion, normalization, augmentation, output classes, loss, learning rate, frozen layers, and checkpoint selection. Keep preprocessing consistent between training and inference.

Evaluation: report per-class precision and recall, a confusion matrix, and an appropriate aggregate metric. Inspect errors by class and acquisition source. For a binary screening scenario, discuss missed positives, false alarms, and threshold choice.

Failure analysis: wrong labels, domain shift, overfitting, similar-looking classes, and patient leakage are hypotheses to investigate. Attribution visualizations can help inspect attention to regions, but do not prove clinical reasoning.

GenAI extension: a language model could explain verified metadata or retrieve approved background material. It should not invent clinical findings from a classifier label. Image classification and medical report generation are different tasks.

Interview template: I worked on [task]. My contribution was [specific work]. We used [method] because [constraint and evidence]. On [split], we measured [actual metrics]. A limitation was [specific issue], and I would next test [concrete improvement].

32. Case study: an internal document assistant

Requirements first: which documents, users, languages, update frequency, query types, latency target, and permission rules? Should it answer from policy only, cite every claim, and abstain when evidence is missing?

Baseline: search plus direct source snippets, then a simple RAG prototype. An LLM's general knowledge alone is unsuitable for current internal policies.

Ingestion: parse authorized documents, preserve structure and dates, identify canonical versions, chunk content, attach document-level permissions, generate embeddings, and index searchable text. Retain exact source spans for citations.

Query path: authenticate, apply access filters, retrieve with dense or hybrid search, rerank suitable candidates, assemble evidence within the token budget, generate a response, and verify citation targets. If evidence conflicts, explain the conflict or escalate rather than guessing.

Evaluation: use held-out real-world-style questions with relevance judgments and supported answers. Measure retrieval recall, answer correctness, citation support, abstention, authorization behavior, latency, and cost per successfully answered query.

Where ordinary ML helps: routing queries, detecting document categories, learning retrieval embeddings, reranking passages, or estimating risk of a failed answer. An embedding model may learn by contrastive training: encourage matched query-passage pairs to be closer than mismatched pairs.

Fine-tuning decision: first isolate the failure. Missing evidence calls for retrieval or content fixes. Wrong consistent formatting may call for prompt or schema improvements, then adaptation if needed. Fine-tuning a generator does not make a stale index current.

Production: version documents and index updates; protect logs and caches; handle provider failure and empty retrieval; monitor user feedback and evaluated quality. Reindex when the embedding model changes.

Interview explanation: I would separate retrieval quality from answer generation so we can diagnose whether the assistant missed the relevant document or misused correct evidence. I would enforce authorization before evidence enters the model context.

Checkpoint: A policy was updated yesterday, but the assistant repeats last month's rule. List the pipeline stages you would inspect.

33. Model selection and systematic debugging

Symptom	Possible causes	First checks
Poor training and validation results	Weak features, bad labels, underfitting, insufficient optimization	Data samples, baseline, loss behavior
Strong training, weak validation	Overfitting, shift, split problems	Learning curves, groups, error slices
Suspiciously excellent test score	Leakage, duplicates, target proxies	Provenance, patient/user overlap, timestamps
Good offline, poor production	Shift, preprocessing mismatch, changed labels	Live schema, transforms, delayed labels
RAG misses needed evidence	Parsing, chunking, embeddings, filters	Retrieved passages and their source text
Evidence is correct, answer is wrong	Context conflict, poor synthesis, instruction failure	Prompt, context order, supported claims

Select against constraints: dataset size and type, target, interpretability, retraining frequency, latency, memory, cost, and error impact. For structured tabular prediction, start with a simple model and tree ensembles. For images or language, pretrained representations are often useful. Compare on a valid evaluation, not popularity.

Hallucination: generated content that is false or unsupported in the relevant setting. Causes include missing knowledge, ambiguous context, learned false associations, and generation behavior. Mitigations include better evidence, grounded instructions, validated tools, task-specific checks, and appropriate abstention. None is a universal guarantee.

Ablation: remove or change one component to measure its contribution. Compare retrieval with and without reranking, or a classifier with and without a feature group. Keep evaluation conditions consistent.

Do not hide uncertainty: explain what the evidence establishes and what it does not. A statistically better score may not justify extra production cost; a small holdout may not establish a stable improvement.

Checkpoint: The client says use the biggest model available. How would you respond in terms of requirements, baselines, evaluation, and operational tradeoffs?

34. Interview-ready answers: core ML

Q1. What is machine learning? Machine learning fits patterns from examples so a model can make useful predictions on new inputs. We define a task, choose features and a learning objective, fit parameters on training data, and evaluate on independent data. The important result is generalization, not simply a low training error.

Q2. Classification versus regression? Classification predicts categories; regression predicts numerical quantities. Predicting whether an order is delayed is classification. Predicting its delay in minutes is regression. Classifiers may output numerical probabilities, but the prediction target is still a category.

Q3. What is overfitting, and how do you handle it? Overfitting occurs when a model learns training-specific noise or patterns that do not generalize. I would confirm it with training and validation behavior and inspect split quality. Depending on the cause, I would use representative data, regularization, simpler models, valid augmentation, or early stopping, then compare on validation data.

Q4. What is data leakage? Leakage lets training or model selection use information unavailable in the intended prediction setting. Examples include future-derived features, preprocessing fitted on all data, and the same patient appearing in both partitions. I prevent it with a deployment-aligned split and transformations fitted only inside training folds.

Q5. Precision versus recall? Precision asks how many predicted positives are correct; recall asks how many actual positives are detected. When missed fraud is costly, recall matters, but higher recall may add false alarms. I would choose a threshold on validation data based on business costs and review capacity.

Q6. Why random forest instead of one tree? A deep tree can vary greatly with the sample. Random forests train diverse trees using bootstrap samples and random feature subsets, then aggregate them. This often reduces variance. The tradeoffs include a larger model and less direct interpretation, so I would compare validation quality and inference cost.

Practice: speak each answer once from memory; then add one example from an actual project. Avoid inventing results to make the answer sound stronger.

35. Interview-ready answers: ML into GenAI

Q7. Backpropagation versus gradient descent? Backpropagation calculates how each trainable parameter affects the loss using the chain rule. Gradient descent is an optimization rule that updates parameters using those gradients. We first compute predictions and loss, then gradients, then an optimizer update.

Q8. Why transfer learning? A pretrained model can provide useful representations when target data or compute is limited. I can freeze its backbone and train a new head, or fine-tune selected layers. I would check whether the source representations transfer to the target domain and whether adaptation overfits or damages useful behavior.

Q9. What is self-attention? Self-attention lets each position combine information from allowed positions in the same sequence. Learned query and key projections produce relevance scores; normalized weights combine value vectors. In a causal language model, masking prevents future-token access. It is information mixing, not a factuality guarantee.

Q10. How does an LLM learn without hand-labeled text? For next-token pretraining, text provides its own targets: earlier tokens are input and the following token is the target. The model minimizes prediction loss across many such examples. Later instruction or preference training can adapt behavior; predicting text well does not by itself guarantee helpful or true answers.

Q11. RAG versus fine-tuning? RAG supplies retrieved evidence during inference. Fine-tuning changes learned parameters or adapters. I would use RAG for current or private source-backed information, and consider fine-tuning for persistent behavior or task performance with good labeled examples. They can be combined, and both need independent evaluation.

Q12. How would you evaluate a document chatbot? I would build held-out representative queries and judge retrieval and generation separately. I would measure relevant-passage retrieval, correctness, groundedness, citation support, abstention, and access control. I would also track latency and cost, use human-reviewed examples to calibrate automated judging, and inspect failure categories.

Practice: answer follow-ups: why not just add all documents to the prompt; why not lower temperature; why not train from scratch?

36. Common traps and precise replacements

Avoid saying	Say instead
Accuracy is always the best metric	Choose metrics and thresholds from class balance and error costs
80/20 is the correct split	Choose a split that reflects new production inputs
Normalize before splitting	Fit learned preprocessing on training data only
More layers always improve results	Compare capacity, data, optimization, and validation quality
Backpropagation updates the weights	Backprop calculates gradients; the optimizer updates weights
Random forests cannot overfit	Ensembles reduce some variance but still need validation
PCA keeps the best predictive features	PCA retains high-variance directions, not necessarily target-relevant ones
Embeddings store exact meanings	Embeddings encode learned relationships in a vector space
An LLM permanently learns from each chat	Ordinary inference changes context, not model weights
Temperature zero prevents hallucinations	Decoding controls randomness, not the truth of claims
RAG eliminates hallucinations	RAG supplies evidence; retrieval and synthesis can still fail
Fine-tuning adds a live factual database	Adaptation changes behavior or knowledge imperfectly; fresh facts need a suitable source
Valid JSON means a correct answer	Check schema, facts, permissions, and business meaning separately
A model explanation proves its reasoning	Generated explanations require independent verification

When you do not know: say what you know, identify the uncertain part, and describe how you would verify it. For example: I understand the purpose of this service, but I have not configured it in production. I would validate the access rules and monitoring in a small deployment first.

Project honesty: distinguish team work, your own contribution, classroom evaluation, and production experience. Interviewers can assess a clearly explained limited project more easily than exaggerated claims.

Checkpoint: Give a corrected explanation for any three traps without looking at the second column.

Closed-notes recall test

Allow 45-60 minutes. Answer aloud or write short explanations. No code is required. Score each: 0 = missing or wrong; 1 = partly correct; 2 = accurate with an example or tradeoff. Total 60. Use the score to find gaps, not to predict interview success.

1. Distinguish AI, ML, deep learning, and GenAI.
2. What differs between supervised and self-supervised learning?
3. Distinguish a feature, a parameter, and a hyperparameter.
4. Why are training, validation, and test data separated?
5. Why should MRI slices be split by patient when possible?
6. How can standardization cause leakage?
7. Which models are particularly sensitive to feature scaling?
8. Explain underfitting versus overfitting using learning behavior.
9. How do L1 and L2 regularization differ?
10. Calculate precision and recall for TP=30, FP=10, FN=20.
11. Why is accuracy misleading when positives are very rare?
12. What changes when a classification threshold is lowered?
13. How do MAE and RMSE respond differently to large errors?
14. Why can R-squared be negative?
15. Why is logistic regression a classification model?
16. Compare bagging, random forests, and boosting.
17. What is the central assumption in Naive Bayes?
18. Why does PCA not guarantee better classification?
19. Distinguish epoch, batch, and optimizer step.
20. Why do neural networks need nonlinear operations?
21. Distinguish backpropagation from an optimizer update.
22. Distinguish a CNN kernel, stride, and pooling.
23. What changes when a pretrained backbone is fine-tuned?
24. Distinguish token IDs, token embeddings, and contextual representations.
25. Explain query, key, value, and causal masking.
26. How does next-token pretraining create its labels?
27. Distinguish full fine-tuning, LoRA, and quantization.
28. Why does lower temperature not guarantee truth?
29. When would you choose RAG over fine-tuning?
30. How do you evaluate retrieval and generated answers separately?

Review order: first correct leakage, metric, and learning errors; then attention and adaptation; then refine project explanations. Answer keys follow.

Recall answer key: 1-15

- 1.** AI is the broad field; ML learns patterns from examples; DL uses multilayer neural networks; GenAI generates content. Modern GenAI often uses DL, but these labels describe overlapping scope and purpose.
- 2.** Supervised learning uses provided targets. Self-supervised learning derives targets from input data, such as a following token or a masked segment.
- 3.** A feature is an input signal; a parameter is learned, such as a weight; a hyperparameter configures learning, such as tree depth or learning rate.
- 4.** Training fits parameters; validation guides choices; test data estimates final selected performance independently. Repeated test-driven tuning breaks independence.
- 5.** Slices from one patient are dependent and may be nearly identical. Keeping patients within partitions avoids evaluating recognition of patients seen in training.
- 6.** Fitting mean and standard deviation on all data exposes held-out distribution information. Fit on training data and reuse those statistics for held-out transformations.
- 7.** KNN, kernel SVMs, regularized linear models, and neural optimization are often sensitive. Ordinary tree splits generally do not require scaling.
- 8.** Underfitting often gives poor training and validation performance. Overfitting often gives strong training but weak validation performance. Audit data and shift before concluding.
- 9.** L1 penalizes absolute weights and can induce sparsity; L2 penalizes squared weights and shrinks coefficients more smoothly. Strength affects the bias-variance tradeoff.
- 10.** Precision = $30/40 = 75\%$. Recall = $30/50 = 60\%$. Precision concerns predicted positives; recall concerns actual positives.
- 11.** Predicting the majority class can achieve high accuracy while missing nearly all positives. Use per-class results and metrics matching error costs.
- 12.** More examples become positive; recall and false-positive count cannot decrease on the same scored dataset. Precision may change in either direction depending on the added cases.
- 13.** RMSE squares errors before averaging and so emphasizes large errors more strongly. MAE has a linear absolute-error penalty.
- 14.** A model can have greater squared error than predicting the evaluation-set mean. That yields a negative R-squared when target variance is nonzero.
- 15.** It models class probability with a sigmoid or softmax transformation; its target is categorical. A numerical probability is not a continuous regression target.

Recall answer key: 16-30

- 16.** Bagging aggregates models fitted on resampled data. Random forests add random feature selection to bootstrapped trees. Boosting adds learners sequentially to improve a loss-related objective.
- 17.** Features are treated as conditionally independent given the class. The assumption can be imperfect while the classifier remains useful.
- 18.** PCA chooses high-variance directions without consulting labels. Predictive signal may occupy a low-variance direction that compression discards.
- 19.** An epoch is one pass through training data; a batch is a group processed together; a step is an optimizer update. Accumulation can use several batches per step.
- 20.** A composition of affine layers alone is another affine transformation. Nonlinear operations enable more expressive input-output relationships.
- 21.** Backprop calculates gradients via the chain rule. The optimizer turns gradients and its state into parameter updates.
- 22.** A kernel is a learned filter; stride controls placement spacing; pooling summarizes local responses and reduces resolution. They serve different roles.
- 23.** Some or all pretrained weights are updated for the target objective. Frozen feature extraction changes only the new trainable components.
- 24.** IDs are discrete tokenizer symbols; embeddings map IDs to vectors; later contextual representations depend on surrounding inputs. Sentence embeddings are another task-oriented representation.
- 25.** Queries compare with keys to create attention weights that mix values. Causal masking blocks future-token access for each prediction position.
- 26.** The text sequence provides targets: predict each next token using preceding tokens. No human needs to label every token pair separately.
- 27.** Full fine-tuning updates the backbone broadly. LoRA freezes original matrices and learns small low-rank updates. Quantization reduces numerical precision and storage; it is not itself fine-tuning.
- 28.** Temperature changes the sampling distribution. If the model's highest-scoring continuation is false, more deterministic selection can repeat that false continuation.
- 29.** Choose RAG when answers depend on current, private, or attributable documents. Fine-tuning is better motivated by recurring behavior or task-performance gaps with suitable examples.
- 30.** Evaluate retrieval using relevance and ranking metrics. Evaluate answers for correctness, groundedness, citations, completeness, and abstention; also measure permission behavior, latency, and cost.

One-page interview cheat sheet

ML workflow: define decision and costs; audit data; design split; build baseline; fit preprocessing on training data; train; validate and tune; test final choice; deploy and monitor.

Learning: supervised uses task labels; unsupervised discovers structure; self-supervised derives labels from data; reinforcement learning uses reward signals from actions.

Evaluation: accuracy can hide imbalance. Precision = $TP/(TP+FP)$. Recall = $TP/(TP+FN)$. F1 = $2PR/(P+R)$. MAE is average absolute error; RMSE emphasizes large errors. Threshold choice and probability calibration are separate from ranking.

Data discipline: group repeated entities; respect time order; fit transformations within training folds; audit duplicates and target proxies. Test scores should not guide repeated tuning.

Model choice: linear/logistic for interpretable baselines; trees for nonlinear feature interactions; forests for variance reduction; boosting for sequential loss improvement; KNN for meaningful proximity; Naive Bayes for simple probabilistic text baselines; PCA for variance-based compression.

Neural training: forward, loss, gradients, update. Backprop calculates gradients; optimizer updates weights. Activation adds nonlinearity. Learning rate controls step size. Epoch, batch, and step are distinct. Regularization and early stopping require validation.

Transfer learning: frozen backbone plus new head is feature extraction; train backbone layers to fine-tune. Watch domain shift and overfitting.

LLM path: tokens; embeddings and position; attention and feed-forward blocks; logits; next-token probabilities; decoding. Pretraining predicts text; SFT teaches examples; preference training favors desired responses.

Attention: learned Q/K compare relevance; weights mix V. Causal masks prevent future-token leakage. Attention is not factual verification.

Adaptation: prompts change context; RAG supplies evidence; fine-tuning changes weights; LoRA learns low-rank updates; quantization reduces precision. They can be combined.

Reliability: evaluate retrieval and generation separately. Verify citations, handle missing evidence, enforce access before retrieval, constrain tools, monitor versions, drift, latency, and cost.

Interview answer: definition, mechanism, example, limitation. **Project answer:** problem, architecture, own contribution, decision evidence, actual metrics, limitations, next improvement.

Glossary and mastery check

Term	Concise meaning
Baseline	Simple reference approach to beat
Label	Desired training outcome
Feature	Input signal used by a model
Inference	Applying a trained model to inputs
Generalization	Useful behavior on intended new inputs
Leakage	Improper information contaminating training or selection
Calibration	Predicted probabilities matching observed frequencies
Hyperparameter	Configuration selected around learning
Embedding	Learned numerical representation
Latent space	Internal representation space
Token	Unit represented by a tokenizer identifier
Logit	Score before probability normalization
Context window	Token span available to a model computation
Fine-tuning	Further training a pretrained model
PEFT	Adaptation using relatively few trainable parameters
RAG	Generation conditioned on retrieved information
Reranker	Model refining candidate relevance order
Hallucination	False or unsupported generated content
Groundedness	Support for an answer in supplied evidence
Drift	A change in data or relevant predictive relationships
Ablation	Controlled removal or change of a component

You are ready to move from concepts to implementation when you can:

- Design a valid split for ordinary, time-series, and grouped data.
- Explain a confusion matrix and choose a metric from error costs.
- Describe at least one linear, tree, distance, and neural method.
- Explain one training iteration without confusing loss with a metric.
- Connect vectors and probability to embeddings and next-token prediction.
- Explain causal attention and the Transformer block at a high level.
- Choose prompting, RAG, or fine-tuning for a concrete failure.
- Discuss your own project using actual evidence and limitations.

Next implementation stage: preprocessing pipeline; simple classifier; evaluation and error analysis; transfer-learning experiment; small evaluated RAG pipeline. These are follow-on tasks, not prerequisites for reading this guide.

References and further reading

The explanations and examples are original educational summaries. These primary documentation pages and research papers support the technical distinctions and provide routes for deeper study. Service interfaces and library behavior can change; check the relevant documentation when you begin implementation.

[1] scikit-learn: Common pitfalls and recommended practices. Preprocessing consistency and leakage. https://scikit-learn.org/stable/common_pitfalls.html

[2] scikit-learn: Cross-validation. Split design, grouped samples, and time-aware evaluation. https://scikit-learn.org/stable/modules/cross_validation.html

[3] scikit-learn: Metrics and scoring. Classification, regression, and clustering measures. https://scikit-learn.org/stable/modules/model_evaluation.html

[4] PyTorch: Optimizing Model Parameters. Training loop, losses, and optimization. https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

[5] Vaswani et al. (2017): Attention Is All You Need. Attention and the original Transformer. <https://arxiv.org/abs/1706.03762>

[6] Ouyang et al. (2022): Training language models to follow instructions with human feedback. A classic SFT and RLHF pipeline. <https://arxiv.org/abs/2203.02155>

[7] Hu et al. (2021): LoRA: Low-Rank Adaptation of Large Language Models. Frozen backbone and low-rank updates. <https://arxiv.org/abs/2106.09685>

[8] Ho et al. (2020): Denoising Diffusion Probabilistic Models. A foundational diffusion formulation. <https://arxiv.org/abs/2006.11239>

[9] Rombach et al. (2022): High-Resolution Image Synthesis with Latent Diffusion Models. Diffusion in compressed representations. <https://arxiv.org/abs/2112.10752>

[10] Dettmers et al. (2023): QLoRA: Efficient Finetuning of Quantized LLMs. Quantized base models and low-rank adaptation. <https://arxiv.org/abs/2305.14314>

[11] Lewis et al. (2020): Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Combining learned and retrieved knowledge. <https://arxiv.org/abs/2005.11401>

[12] Rafailov et al. (2023): Direct Preference Optimization. Preference learning without the classic explicit reward-model/RL loop. <https://arxiv.org/abs/2305.18290>